

Docker Architecture & Core Concepts

بنية دوكر والمفاهيم الأساسية.

Ship Apps
Anywhere! ❤️

1 Container vs Image

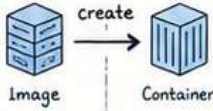
ال Container وال Image والفرق الجوهرى بينهما.

Image (صورة)

- قالب . read-only . يحتوي على كل حاجة . مطلوبة لتشغيل التطبيق .
- يشبه blueprint أو template .
- لا يتم تغييره عند التشغيل .
- يُستخدم لإنشاء Containers .
- ubuntu:22.04 , node:18

Container (حاوية)

- نسخة ال OS بها صمد (قيد التشغيل) من Image .
- تعتمد على read-write قوة image .
- معزول (isolated) عن باقي ال .
- يمكن start / stop / stopl .
- تطبيقك شغال جواها .



Memory Trick:

Image = Class (تعريف)
Container = Object (كائن من الكلاس)

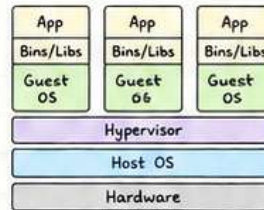
2 Hypervisor & Virtual Machines

VS
Containerization

Virtual Machines (VMs)

- تشترك ال OS فيها OS كامل خاص بها .
- Hypervisor فوق image . (VMware , Hyper-V مثل)
- ثقيلة (Heavy) وتستهلك موارد (CPU , RAM , Disk) أكثر .
- إقلاع أبطأ .

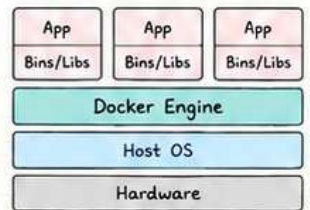
VMs Architecture



Containerization

- تشارك ال . Kernel . OS مع ال host .
- Engine على Docker .
- خفيفة (Lightweight) وسريعة .
- تستخدم موارد أقل .
- إقلاع سريع . جدًا .

Containers Architecture



Key Idea: VMs = Each with its own OS
Containers = Share the OS



3 Docker Engine Components

مكونات ال Docker Engine .



Note: ال الخلفية لا يتحدث مباشرة مع ال Daemon . بل من خلال ال CLI ال REST .

4 Docker Registry & Docker Hub

Docker Registry

- مستودع (repository) لتخزين ال images .
- يمكن يكون خاص (Private) داخل الشركة .
- يحتوي على images يمكنك (push و pull) .
- registry.mycompany.com مثل

Docker Hub



- أشهر registry على الإنترنت .
- يحتوي على ملايين ال images ال الجاهزة .
- يمكنك رفع صورك الخاصة (push) .
- hub.docker.com .

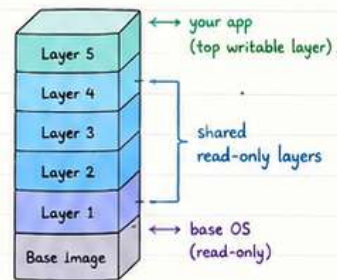


Memory Trick:

Registry = مكان التخزين
Hub = أشهر مكان عام للتخزين

5 Layers (الطبقات)

- ال الخلفية مكون من طبقات علي (Layers) .
- كل طبقة تمثل تغيير أو إضافة في Dockerfile .
- الطبقات تكون read-only ومشاركة بين ال images .
- عند تغيير بسيط , يتم إضافة طبقة جديدة فقط .
- هذا يجعل التخزين والبناء أسرع وأصغر حجمًا .



Think:

كل طبقة = خطوة
في Dockerfile

6 Storage Drivers

- Storage Driver هو . Storage . images , layers , ال containers .
- يقدم دعم عدة snapshor حسب نظام التشغيل ال system .
- أهم ال drivers :

Driver	Description	Best For
overlay2	الأكثر استخدامًا في Linux . union filesystem يستخدم .	Linux (RHEL/CentOS/Ubuntu)
btrfs	snapshots , compression . يقدم ميزات متقدمة مثل .	Linux
devicemapper	قديم لكنه قوي , يعتمد على LVM .	Linux (older systems)
aufs	قديم . Union filesystem .	Linux (deprecated)
windowsfilter	المستخدم في Windows .	Windows

How it works?

- 1) Image pulled → layers stored by storage driver.
- 2) Container created → thin writable layer added on top.
- 3) All data managed efficiently.

Exam Tip

- Image = read-only template.
- Container = running instance.
- Containers are lightweight because they share the OS kernel and use layered architecture.



2 Dockerfile



الطريقة التي يكتب فيها تعليمات لبناء Docker Image - خطوة

Think:

Dockerfile = Recipe

Image = Cake



Dockerfile Instructions

FROM

- تحدد Base Image التي هنتبني عليها
- FROM python:3.11-slim

RUN

- تشغل أوامر في ال Image أثناء البناء
- RUN apt-get update && apt-get install -y curl

COPY

- تنسخ ملفات أو فولدرات من جهازك إلى ال Image
- COPY . /app

ADD

- زي COPY لكن ممكن كمان يفتك أرشيفات (zip, tar) و يلاصها
- ADD file.tar.gz /app/

CMD

- تحدد الأمر الافتراضي اللي هيشغل لما الكونتير يبدأ
- CMD ["python", "app.py"]

ENTRYPOINT

- تحدد أمر ثابت دائماً بتنفيذ عند تشغيل الكونتير
- ENTRYPOINT ["python"]

EXPOSE

- توضح البورت اللي هتستخدمه التطبيق (مسي بيقتحه فعلياً)
- EXPOSE 8080

ENV

- تعريف متغير بيته داخل ال Image
- ENV APP_ENV=production

ARG

- متغير وقت البناء فقط (Build-time)
- ARG VERSION=1.0

WORKDIR

- تحدد الدليل الافتراضي اللي هتشغل منه الأوامر
- WORKDIR /app

USER

- تشغل الأوامر كمستخدم معين (لأمان أعلى)
- USER nonroot

CMD vs ENTRYPOINT

CMD

- Default command or arguments
- command line ممكن إل من
- في ال Docker مع ممكن تستخدم واحد بس

ENTRYPOINT

- ثابت
- إميش بيتغير (إلا لو (override)
- ممكن ك CMD كد تستخدم مع arguments

Examples

1) Using CMD (overridable)

Dockerfile
FROM alpine
CMD ["echo", "Hello"]

Build & Run

\$ docker build -t myapp .
\$ docker run myapp → Hello

Override:

\$ docker run myapp echo Bye → Bye

2) Using ENTRYPOINT (fixed)

Dockerfile
FROM alpine
ENTRYPOINT ["echo"]
CMD ["Hello"]

Build & Run

\$ docker build -t myapp .

\$ docker run myapp → Hello

\$ docker run myapp Bye → Bye



Rule of Thumb:

CMD = What to run

ENTRYPOINT = How to run



Layer Caching

- Docker بيبنى Image على شكل طبقات (Layers) كل طبقة بتخزن وتعاد استخدامها لو مافيش تغيير.

إزاي بيشتغل؟

- Layer 5: CMD
- Layer 4: COPY . /app
- Layer 3: RUN pip install -r req.txt
- Layer 2: WORKDIR /app
- Layer 1: FROM python:3.11-slim

لو غيرت في طبقة معينة كل الطبقات اللي بعدها هتبنى من جديد

إزاي نحسن ال Caching؟

- رتب التعليمات من الأقل تغيير للأكثر تغيير

مثال:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

- افصل نسخ dependencies عن كود المشروع
- لا تستخدم / في إبي البداية
- استخدم .dockerignore لتقليل الملفات المرسلة
- قلل عدد ال RUN قدر الإمكان
- استخدم صور أصغر (slim/alpine)

أسرع build → أقل rebuild = الهدف

Multi-stage Builds

لتصغير حجم ال Image باستخدام مراحل بناء متعددة

الفكرة



المميزات

- بدون نهاسي أصغر بكثير
- لا يحتوي على أدوات بناء
- أكثر أماناً
- أسرع في النقل و التشغيل

مثال (Go App)

```
# ----- Stage 1: Build -----
FROM golang:1.21 AS builder
WORKDIR /src
COPY . .
RUN go build -o myapp

# ----- Stage 2: Runtime -----
FROM alpine:3.19
WORKDIR /app
COPY --from=builder /src/myapp .
CMD ["/myapp"]
```

multi-stage



1.2 GB

multi-stage



25 MB

أصغر بكثير!

.dockerignore

node_modules/	→	تجاهل node_modules
.git/	→	تجاهل مجلد .git
.env	→	تجاهل ملفات .env
*.log	→	تجاهل أي ملف .log
dist/	→	تجاهل مجلد dist
Dockerfile	→	تجاهل مجلد Dockerfile
.dockerignore	→	تجاهل .dockerignore

نصيحة

- دائماً إحتط الملفات غير الضرورية في .dockerignore خاصة لو فيها بيانات حساسة أو ملفات كبيرة.



Dockerfile Cheat Sheet (Quick View)

FROM	Base image	ENTRYPOINT	Fixed command
RUN	Run commands	EXPOSE	Document port
COPY	Copy files	ENV	Environment variable
ADD	Copy + extras	ARG	Build-time variable
CMD	Default command	WORKDIR	Set working dir
		USER	Run as specific user

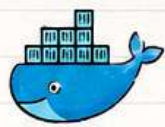
Don't Forget

- رتب التعليمات صح
- تأكد من layer = layer
- استخدم multi-stage
- اعتمد علي .dockerignore

Happy Dockering! 🐳

Memory Tricks

FROM	→	Foundation (القاعدة)
COPY	→	Copy your files
ADD	→	Add more (or download)
RUN	→	Run commands
CMD	→	Command by default
ENTRY	→	Entry (the fixed door)
EXPOSE	→	Expose your app
ENV	→	Environment
ARG	→	Argument (build-time)
WORKDIR	→	Work directory
USER	→	User (security first)



Docker

٣. أوامر دوكر الأساسية

Docker

يصنع ، يشغل ،
ينقل تطبيقاتك بسهولة
وفي أي مكان

Docker = Containerization Platform

دوكر يسمح لنا نحزم التطبيق مع كل dependencies في وحدة واحدة
اسمها Container ونشغلها في أي بيئة بنفس الطريقة.

ال أوامر الأساسية

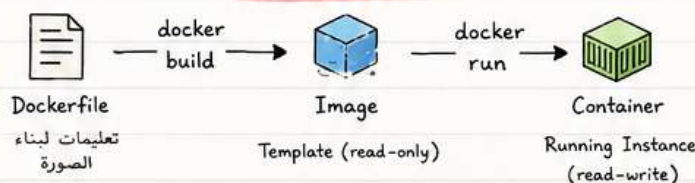
ملاحظات	مثال	الشرح	الأمر
(اسم للصورة) tag : -t : من نفس المكان	docker build -t myapp:1.0	بيني Image جديد من Dockerfile (يقرأ التعليمات ويصنع صورة)	docker build
(خلفية) detached : -d port mapping : -p	docker run -d -p 8080:80 myapp:1.0	يشغل Container جديد من Image (Create + Start)	docker run
يجلب آخر نسخة	docker pull nginx:latest	Image من Docker Hub	docker pull
لازم تسجيل دخول أولاً (docker login)	docker push myuser/myapp:1.0	يرفع Image إلى Image	docker push
(حتى المتوقعة) all : -a	docker ps docker ps -a (جميعها)	Containers المشتغلة حالياً	docker ps
REPOSITORY, TAG, SIZE ...	docker images	يعرض Images الموجودة محلياً	docker images
تشوف (Logs) follow : -f	docker logs <container_id> docker logs -f <id> (يتابع)	يعرض Logs (سجلات) Container	docker logs
تفاعل تفاعلي : -it (interactive terminal)	docker exec -it <id> bash (يدخلك على ال shell)	يشغل أمر داخل Container شغال	docker exec
يرسل إشارة توقف آمنة	docker stop <id أو name>	يوقف Container شغال	docker stop
حذف إجباري : -f	docker rm <id أو name> docker rm -f <id> (قوة)	يحذف Container (المتوقف فقط)	docker rm
-f : force	docker rmi <image_id> docker rmi -f <id>	يحذف Image (لازم ما يكون مستخدم)	docker rmi
مفيد للتشخيص والتصحيح	docker inspect <id/image>	يعرض تفاصيل كاملة (JSON) عن Image أو Container	docker inspect
يعرض بشكل مباشر (Real-time)	docker stats (أو container محدد)	يعرض استهلاك موارد (Containers CPU, MEM...)	docker stats
يشبه أمر top في لينكس	docker top <id>	يعرض العمليات (Processes) الجارية داخل Container	docker top
صيغة : docker cp <src> <dest>	Host إلى Container docker cp file.txt <id>:/app/ image إلى Container docker cp <id>:/app/file.txt ./	ينسخ ملفات/مجلدات بين Container	docker cp
مفيد لعمل نسخة احتياطية (Backup Image)	docker commit <id> myapp:backup	يحفظ حالة Container كـ Image جديد	docker commit

Memory Tricks

Build → ابني صورة
Run → شغل الحاوية
Pull → أنزل من الإنترنت
Push → ارفع للصحاب (Hub)
PS → Process Containers (show)
Images → الصور
Logs → السجلات
Exec → Execute (داخل داخية)
Stop → قف (توقف)
Rm → Remove Container
Rmi → Remove Image

Inspect → التفاصيل كـ
Stats → الإحصائيات
Top → القيمة ، (أعلى العمليات)
Cp → Copy (نسخ)
Commit → احفظ كصورة جديدة

Docker Flow



Container Lifecycle



Tips

- استخدم -d دائماً للتشغيل في الخلفية.
- يعرض docker ps -a جميع الحاويات.
- Logs هي أول مكان تبحث فيه عند حدوث خطأ.
- نظف دائماً ما لا تحتاجه :
docker system prune (يحذف غير المستخدم)

تذكر دائماً : الصورة (Image) = قالب ثابت ، الحاوية (Container) = نسخة شغالة من هذا القالب ..

4 Networking

Connect Containers to the World

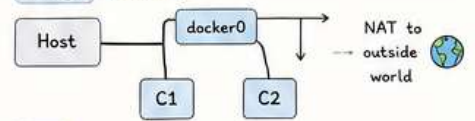
Think:

Networking is how containers talk to each other, to the host, and to the outside world.

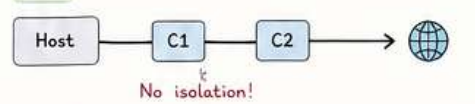
1 Docker الشبكات في

النوع	الاستخدام	الوصف	الاتصال بالخارج	متى استخدمه؟
bridge	للتطبيقات العامة (Default)	كل Container يباذ IP خاص من الشبكة الداخلية (Bridge) وال containers بتقدر تتكلم مع بعض ومع ال host.	نعم (NAT) ✓	معظم الحالات. تطبيقات Web, Databases, APIs.
host	أداء عالي (High Performance)	ال Container يشارك نفس شبكة ال Host مباشرة (مافيش عزل شبكي).	نعم (نفس ال host) ✓	لأداء عالي جداً أو تطبيقات تحتاج وصول مباشر للشبكة.
none	عزل تام (No Network)	ال Container ما عنده أي شبكة نهائياً.	لا ✗	حالات أمنية خاصة أو تطبيقات معزولة.
overlay	التواصل بين عدة Docker Hosts	شبكة مخصصة للتواصل بين containers على عدة Hosts (Swarm Mode).	نعم (حسب التهيئة) ✓	أنظمة موزعة (Microservices) باستخدام Docker Swarm.

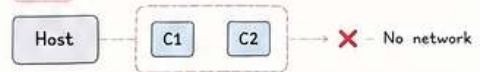
bridge (default)



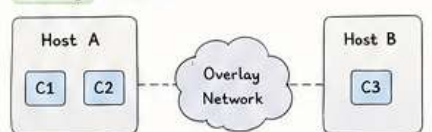
host



none



overlay (swarm)



Works across multiple hosts

2 Port Mapping (-p)

عشان نفتح بورت من ال host ونربطه ببورت داخل ال container.

```
docker run -d -p 8080:80 --name web nginx
```

Port على host (8080)

Port داخل container (80)

أمثلة:

```
-p 80:80 # http
-p 443:443 # https
-p 127.0.0.1:3306:3306 # localhost من ال
-p 3000:3000 -p 3001:3000 # port لنفس مختلفة app
```

يعني إيه؟

أي طلب جاي على ال host من البورت 8080 هيتوجه لل container على البورت 80

ملاحظات:

- ✓ ممكن تبقى المابنج لأي port على ال host.
- ✓ لو مذكرتش ال host port, Docker, مينتخار واحد عشوائي.
- ✓ استخدم docker ps لعرض المابنج.

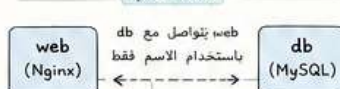
3 Docker DNS - containers talk by name

عشان عندنا داخلي تلقائي لكل ال containers على نفس الشبكة. بيس Docker.

How it works?

- ال container يباذ اسم (--name) أو اسم تلقائي.
- Docker DNS بيخزن IP - IP mapping.
- 3- لما container يحاول يتواصل مع اسم (db) ال DNS بيترجم الاسم ال IP الصحيح.
- بعد كده يحصل الاتصال مباشرة.

مثال عملي



```
# web web container
mysql -h db -u user -p
db مش محتاج يعرف IP بتاع
```

DNS Records (داخل الشبكة)

```
web → 172.18.0.2
db → 172.18.0.3
redis → 172.18.0.4
```

يتم التحديث تلقائياً

قاعدة ذهبية

تأكد إن ال B. H. N. O. التي عابزم بتكلموا مع بعض بيقوا على نفس الشبكة!

4 Create a Custom Network & Connect Containers

1 Create network

```
docker network create mynet
```

(bridge: الافتراضي: المكنج)

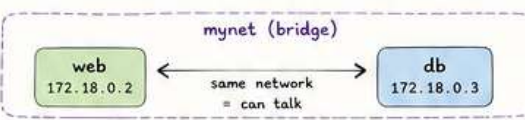
Check

```
docker network ls
docker network inspect mynet
```

2 Run containers on that network

```
docker run -d --name web --network mynet nginx
```

```
docker run -d --name db --network mynet mysql:8
```



لما يكونوا على نفس ال network. يتكلموا مع بعض باستخدام الاسم (web, db) مش ال IP.

3 Connect existing container to network

```
docker network connect mynet redis
```

redis
172.18.0.4

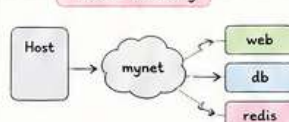
Disconnect

```
docker network disconnect mynet redis
```

Useful Commands

```
docker network ls # عرض الشبكات
docker network inspect <net> # تفاصيل الشبكة
docker network rm <net> # حذف شبكة
docker network connect <net> <container> # ربط
docker network disconnect <net> <container> # فصل
```

Visual Summary



- ✓ Same network = can talk
- ✗ Different network = can't
- ★ Overlay = across hosts
- 🌐 Port mapping = expose to outside

Memory Trick

B. H. N. O

- B = Bridge (default)
- H = Host (no isolation)
- N = None (no network)
- O = Overlay (across hosts)



Docker



5. Volumes والبيانات

ال Container يمكن يتوقف أو يتحدد لكن البيانات تفضل محفوظة باستخدام Bind Mounts أو tmpfs أو tmpfs

Goal:

نحافظ على البيانات ونشاركها بأمان بين Containers



1 Volumes vs Bind Mounts vs tmpfs

	Volumes (Managed by Docker)	Bind Mounts (Host Path)	tmpfs (In-Memory)
التعريف	مساحة تخزين تدار بواسطة Docker خارج ال Container	ربط مسار من جهاز ال Host داخل ال Container	تخزين في الذاكرة (RAM) مؤقت
المكان	على Host في مسار خاص بـ Docker (/var/lib/docker/volumes/)	أي مسار على ال Host تحدده	داخل RAM فقط
الادارة	يُنشأ ويُدار بـ Docker CLI	تديره أنت (ملفات النظام)	يُدار بواسطة Docker وقت التشغيل
الأمان	أكثر أماناً ومعزول عن ملفات النظام	قد يعرض ملفات النظام للخطر ⚠️	لا يُخزن على القرص (أكثر أماناً مؤقتاً)
الاستخدامات	قواعد بيانات، ملفات تطبيقات ثابتة	التطوير (dev)، تعديل الكود مباشرة، ملفات إعدادات خارجية	ملفات مؤقتة، Caching، بيانات حساسة لا تُحفظ على القرص
السرعة	سريعة	تعتمد على سرعة القرص	(RAM) 🚀
متى تُستخدم؟	في البيئات الإنتاجية (Production)	في التطوير أو عند الحاجة لمشاركة ملفات معينة من ال Host	عند الحاجة لأداء عالي وبيانات مؤقتة
مثال	-v mydata:/var/lib/mysql	-v /home/ahmed/project:/app	--tmpfs /run:rw,size=64m

2 إنشاء وإدارة Volumes

1) إنشاء Volume

```
docker volume create <volume_name>
```



مثال:

```
docker volume create mydata
```

2) عرض كل Volumes

```
docker volume ls
```

3) تفاصيل Volume

```
docker volume inspect mydata
```

4) حذف Volume

```
docker volume rm mydata
```

(لا يمكن حذف Volume وهو مستخدم) ⚠️

5) حذف غير المستخدم فقط

```
docker volume prune
```

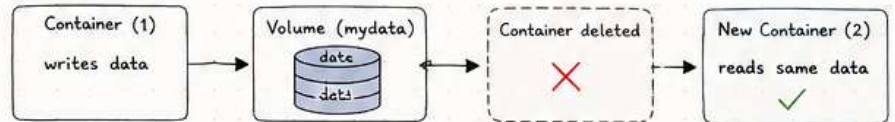
يحذف كل ال Volumes غير المستخدمة 🧹

3 Data Persistence (الحفاظ على البيانات)

الفكرة:

البيانات المخزنة في Volume تبقى حتى إذا تم حذف ال Container ثم إنشاء مرة أخرى.

مثال عملي:



```

# run container with volume
docker run -d --name app1 -v mydata:/data alpine sh -c "echo hello > /data/hello.txt"

# delete container
docker rm -f app1

# run new container with same volume
docker run --name app2 -v mydata:/data alpine cat /data/hello.txt

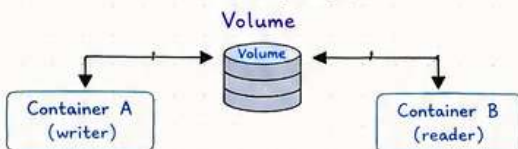
# Output: hello
  
```

الخلاصة:

ال Volume منفصل عن ال Container = البيانات تفضل موجودة. ⭐

4 Volume Sharing

يمكن مشاركة نفس ال Volume بين أكثر من Container (قراءة أو كتابة).



مثال:

```

# Container A (write)
docker run -d --name writer -v shared_data:/data alpine sh -c "while true; do date >> /data/log.txt; sleep 5; done"
  
```

```

# Container B (read)
docker run -it --name reader -v shared_data:/data alpine tail -f /data/log.txt
  
```

(shared_data) (shared_data)

يتم وضعه في مسار /data داخل كل Container



Memory Trick

V = Volume (managed by Docker)
B = Bind (from Host)
T = tmpfs (in RAM / Temporary)

Best Practice

+ استخدم دائماً Volumes في Production.
لا تربط مجلدات حساسة من ال Host إلا عند الحاجة.
+ اعمل Backup للبيانات المهمة.
لا تعتمد على tmpfs للبيانات الدائمة (لأنها في RAM).



Quick Cheat Sheet

docker volume create <name>	إنشاء Volume جديد
docker volume ls	عرض كل ال Volumes
docker volume inspect <name>	عرض تفاصيل Volume
docker volume rm <name>	حذف Volume
docker volume prune	حذف غير المستخدم
docker run -v mydata:/path	استخدام Volume داخل Container
docker run -v /host/path:/path	استخدام Bind Mount
docker run --tmpfs /path	استخدام tmpfs

TIP

Database + Volume = Happy Data 🤗



5 Docker Compose

Define & Run Multi-Container Applications

Think:

Compose = Orchestrate locally
Multiple containers as one app

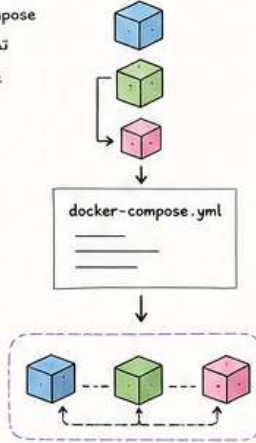
1 ما هو Docker Compose

Docker Compose هو أداة تساعدك تشغيل وتدير تطبيق مكون من أكثر من Container عن طريق ملف واحد YAML اسمه docker-compose.yml

docker-compose.yml

ليه نستخدمه ؟

- تشغيل كل الخدمات بأمر واحد
- تسهيل إدارة التطبيقات المعقدة
- ربط الشبكات Volumes بسهولة
- نفس البيئة لكل أعضاء الفريق



2 docker-compose.yml و بنيته

```
version: "3.9" # version of Compose file
services:
  web:
    image: nginx:alpine
    container_name: web
    ports:
      - "8080:80"
    environment:
      - ENV_VAR=value
    volumes:
      - ./html:/usr/share/nginx/html
    depends_on:
      - db
    networks:
      - mynet
  db:
    image: mysql:8
    environment:
      - MYSQL_ROOT_PASSWORD=123456
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - mynet
networks:
  mynet:
    driver: bridge
volumes:
  db_data:
```

أقسام الملف :

- version
 - Compose file
- services
 - كل خدمة (خدمة) بتشتغلها
- networks
 - تعريف الشبكات المشتركة
- volumes
 - تعريف التخزين الدائم

مهم !

مهم Indentation
YAML يعتمد علي المسافات

3 في الكومبوز

كل خدمة عبارة عن Container لها إعداداتها الخاصة.

النوع	الوصف	مثال
image	الصورة اللي هتشغل منها	image: nginx:alpine
build	تبنى الصورة من Dockerfile	build: ./app
container_name	اسم ثابت للكونتينر	container_name: web
ports	ربط بورت من الـ host	- "8080:80"
environment	متغيرات البيئة داخل container	- ENV=prod
volumes	ربط مجلدات (حجم دائم)	- ./data:/data
networks	الشبكات اللي هينضم لها	- mynet
depends_on	تحديد الخدمات المعتمدة	- db
restart	سياسة إعادة التشغيل	restart: unless-stopped

4 في networks

تسمح لك containers تتواصل مع بعض باسمائها.

الاستخدام	الوصف	driver
الأكثر استخداماً	شبكة داخلية علي نفس الجهاز	bridge (default)
swarm / multi-host	شبكة بين أكثر من Docker Host	overlay
أداء عالي	يستخدم شبكة الـ host مباشرة	host
عزل كامل	بدون شبكة	none



داخل نفس network
containers الـ
تقدر تهتم باسمائها:
db ✓

5 في volumes

بتحفظ بتبيانات احفظها بشكل دائم حتى اتصال الـ container.

النوع	الوصف	مثال
named volume	يتم إدارته بواسطة Docker	db_data:/var/lib/mysql
bind mount	مجلد من جهازك الربط	./data:/data
tmpfs	في الذاكرة فقط (مؤقت)	type: tmpfs

قاعدة ذهبية !

أي بيانات مهمة → احفظها في Volume
مش Container



6 Docker Compose المهمة

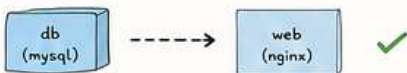
الأمور	الشرح	مثال
docker compose up	تشغيل كل الخدمات (build) لو لازم	up
docker compose up -d	تشغيل في الخلفية (detached)	up -d
docker compose down	إيقاف containers, networks, networks	down
docker compose build	بناء الصور للخدمات	build
docker compose logs	عرض logs للخدمات	logs
docker compose logs -f	عرض logs بشكل مستمر	logs -f
docker compose exec web bash	تشغيل أمر داخل container	exec
docker compose ps	عرض حالة الخدمات	ps

7 في depends_on و التشغيل

يحدد أن خدمة تعتمد علي خدمة أخرى.

services:
web:
 depends_on:
 - db
db:
 image: mysql:8

- هشغل هبى db قبل Compose ✓
- لكن ما بيعملش إن db جاهزة تماماً (بس إنه اشغل)
- في التطبيقات الحساسة استخدم healthcheck أو wait-for-it scripts



8 environment variables و ملفات .env

ملف .env بـ docker-compose.yml

```
services:
  web:
    environment:
      - APP_ENV=production
      - DEBUG=false
    # key: value بالشكل
    environment:
      APP_ENV: production
      DEBUG: "false"
```

(ب) ملف. (موصى به ؟)

```
# .env file
APP_ENV=production
DEBUG=false
DB_USER=admin
DB_PASS=123456
```

الاستخدام في الملف

```
services:
  db:
    environment:
      - DB_USER=${DB_USER}
      - DB_PASS=${DB_PASS}
```

كيف نستخدم .env ؟

Compose تقرأ ملف .env من نفس مجلد
docker-compose.yml

Memory Trick

- التعيين هما services
- الملعب هي Networks
- التخزين هي Volumes
- المدرّب هو Compose

Best Practices

- استخدم .env للبيانات الحساسة
- فصل كل خدمة في network مناسبة
- استخدم volumes لأي بيانات مهمة
- استخدم restart: unless-stopped للإنتاج

Cheat Sheet (Quick View)

- docker compose up -d # run in background
- docker compose down # stop & remove
- docker compose build # build images
- docker compose logs -f # follow logs
- docker compose exec <service> <cmd> # run command

You got
this! 🍷



Docker

7. Best Practices والأمان

ممارسات صحيحة = كوتنيترات آمنة، مستقرة، وسهلة الصيانة

Goal:

نبني تطبيقات Docker
آمنة، خفيفة، واحترافية
جاهزة للإنتاج

1 استخدام images رسيعة وصغيرة



الفكرة:

اختر images رسمية من Docker Hub ويفضل تكون صغيرة (مثل alpine) علشان تقلل الحجم والثغرات الأمنية.

✓ DO

استخدم Official images من جهات موثوقة
اختر نسخ خفيفة (alpine)
استخدم stable stable version
قلل عدد الطبقات في Dockerfile
نظف الكاش والملفات غير الضرورية

✗ DON'T

لا تستخدم images مجهولة المصدر
لا تبني images كبيرة بدون سبب
لا تثبت حزم كثيرة مش محتاجها
لا تترك بيانات حساسة داخل الصورة

لماذا Alpine ؟

	Debian/Ubuntu	Alpine
الحجم	~ 200MB+	~ 5MB - 20MB
Packages	كبير (more)	قليل (minimal)
Security Surface	أكبر	أقل
Boot Time	أبطأ	أسرع
استخدم عندما	تحتاج أدوات كثيرة	تحتاج خفة وأمان

Dockerfile باستخدام سنح Alpine

```

FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]

```

python:3.12-alpine
صغيرة وخفيفة

→ --no-cache-dir
يمنع تخزين الكاش
ويقلل الحجم

نُفلة لتقليل حجم ال Tips

- ✓ استخدم dockerignore. لتجاهل الملفات غير الضرورية
- ✓ ادمج الأوامر في layer واحد قدر الإمكان
- ✓ احذف الكاش: apk add --no-cache (في Alpine)
- ✓ استخدم Multi-stage builds عند الحاجة

2 Don't run containers as root



الفكرة:

تشغيل التطبيق كمستخدم root داخل الكونتنيتر يزيد من خطورة الاختراق. الأفضل تشغيله بمستخدم عادي (غير root) بصلاحيات أقل.

⚠️ الخطر

- لو حصل اختراق داخل الكونتنيتر والمستخدم root، المهاجم ممكن يعمل أي حاجة داخل النظام.
- قد يتم تعديل ملفات النظام أو سرقة بيانات حساسة.
- ممكن يتجاوز العزل بين ال containers.

✓ التوصية

شغل التطبيق بمستخدم غير root وباقل صلاحيات ممكنة (Principle of Least Privilege)

اختر images جاهزة تعمل كمستخدم غير root

بعض ال images الرسمية توفر مستخدم root جاهز:

- node → node كمستخدم
- nginx → nginx كمستخدم
- postgres → postgres يعمل كمستخدم
- mysql → mysql يعمل كمستخدم

كيف تشغيل كمستخدم غير root

Dockerfile: استخدام في (الطريقة 1)

```

FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production

# إنشاء مستخدم غير root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY . .
CMD ["node", "server.js"]

```

adduser في Alpine
يعمل مستخدم بسيط

→ يحدد USER
المستخدم الافتراضي
لتشغيل التطبيق

الطريقة 2: من سطر التشغيل (Runtime)

docker run -d --name myapp --user 1001:1001 myapp:latest

تستخدم UID/GID معيّن من 1001:1001

يمكنك أيضا `ogtee $(id -u):$(id -g)` لمطابقة المستخدم الحالي

تأكد من المستخدم داخل الكونتنيتر

docker exec -it myapp whoami

Output: appuser

Bad
(root)

risk high

risk low

Good
(non-root)

risk low

Quick Cheat Sheet

FROM alpine	استخدم image صغيرة
RUN apk add --no-cache <pkg>	ثبت بدون كاش
RUN adduser -S appuser	أنشئ مستخدم غير root
USER appuser	شغل كمستخدم غير root
docker run --user 1001:1001 img	شغل بمستخدم محدد
docker exec -it <c> whoami	اعرف المستخدم داخل الكونتنيتر

Memory Trick

- A → Alpine (Light)
- L → Less Packages
- P → Performance Better
- I → Image Small
- N → Need Secure
- E → Efficient

"Less is More"

image صغيرة =
أمان أعلى + سرعة أعلى
+ مساحة أقل

الخلاصة: استخدم images رسمية وخفيفة (Alpine) + لا تشغل كمستخدم root = أمان أعلى وأداء أفضل

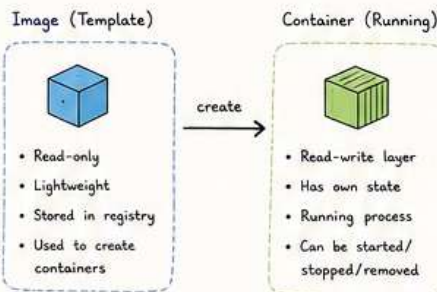


INTERVIEW QUESTIONS

Remember:
Build once,
Run anywhere.

① Q: What is the difference between Docker Image and Docker Container?

A: A Docker Image is a read-only template used to create containers. It is like a blueprint. A Docker Container is a running instance of an image.



Memory Tip:
Image is like a class, Container is an object.

② Q: What is the difference between Container and Docker Container?

A: Containers share the host OS kernel, so they are lightweight and start very fast. VMs include a full OS (guest OS) with hypervisor, so they are heavier.

Feature	Container	Virtual Machine
Architecture	Share host OS	Full guest OS
Isolation	Process level	OS level
Startup Time	Seconds	Minutes
Size	MBs	GBs
Performance	High	Lower
Use Case	Microservices, Dev/Test	Different OS, Strong isolation

Note:
Containers are like customers in one building. VMs are like houses in a separate neighborhood.

③ Q: What is a Dockerfile and what is it made of?

A: A Dockerfile is a text file that contains instructions to build a Docker Image. It is made of instructions (commands) like FROM, RUN, COPY, CMD, etc.

Common Dockerfile Instructions

FROM	→ Base image
RUN	→ Run commands at build time
COPY	→ Copy files from host
ADD	→ Copy + extra features (url, tar)
CMD	→ Default command when container runs
EXPOSE	→ Document the port
ENV	→ Set environment variables
ARG	→ Build-time variables
WORKDIR	→ Set working directory
USER	→ Run as specific user

Memory Trick:
F R C E A W U
FROM, RUN, COPY, CMD, EXPOSE, ARG, WORKDIR, USER

④ Q: What is the difference between CMD and ENTRYPOINT?

A: CMD provides default command and can be overridden. ENTRYPOINT sets the main command and is usually not overridden.

	CMD	ENTRYPOINT
Purpose	Default command	Main command
Override	Can be overridden	Usually not overridden
Multiple CMD	Only last one used	N/A
Use Case	Provide defaults	Define what container does

Example

Dockerfile	Dockerfile
CMD ["nginx", "-g", "daemon off;"]	ENTRYPOINT ["nginx"]
	CMD ["-g", "daemon off;"]

Tip: Use CMD for options, ENTRYPOINT for the app itself.

⑤ Q: How to run a container in the background (detached mode)?

A: Use the -d or --detach flag.

```
docker run -d --name web nginx
```



Note:
Use docker ps to see running containers (without -a).

```
docker ps  
(running only)
```

⑥ Q: What is the difference between docker stop and docker kill?

A: docker stop sends SIGTERM (polite request) and gives time to clean up. docker kill sends SIGKILL (force) immediately.

Command	docker stop	docker kill
Signal	SIGTERM	SIGKILL
Graceful Shutdown	Yes	No
Use When	Normal stop	Force stop
Default Timeout	10 seconds	Immediate

Warning: Prefer docker stop. Use kill only when a container doesn't stop.

⑦ Q: What is Docker Hub and why do we use it?

A: Docker Hub is a cloud registry (storage for images). We use it to:

- Store and share images
- Pull official or community images
- Push our custom images
- Automate deployments (CI/CD)

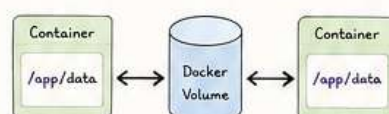


Quick Facts:

- Official images are verified
- Public & Private repositories
- Command: docker push/pull

⑧ Q: What is a Docker Volume and why do we use it?

A: A Docker Volume is a persistent storage managed by Docker. Data in containers is ephemeral (temporary). Volumes help us persist data even if the container is removed.



Benefits

- Data persists after container removal
- Share data between containers
- Better performance than bind mounts
- Easy backup & migration

Memory Tip:
Container is temporary, Volume is permanent.

BONUS CHEAT SHEET

Useful Commands

• docker ps	# List running
• docker ps -a	# List all
• docker images	# List images
• docker run -d --name c1 image	# Run
• docker stop c1	# Stop container
• docker rm c1	# Remove container
• docker rmi image	# Remove image
• docker logs c1	# View logs
• docker exec -it c1 bash	# Access container

Think like Docker:
Build with layers,
Run with containers,
Ship with confidence!



INTERVIEW QUESTIONS

Docker
Advanced
(Senior Level)

1/3

Q1: How do you build the smallest Docker image possible for production?

A:

- Use minimal base image (e.g., alpine).
- Use multi-stage builds.
- Install only what you need.
- Clean up cache and unnecessary files.
- Use .dockerignore.
- Use distroless (no shell) images if possible.

EXAMPLE (Multi-stage)

```
# Build stage
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM node:18-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
CMD ["node", "dist/index.js"]
```



MEMORY NOTE

Small Image = Fast Pull +
Less Storage + Less Attack Surface

Q2: How do you handle secrets in Docker without putting them in the image?

A:

- Never put secrets in Dockerfile, image, or environment variables.
- Use Docker Secrets (Swarm) or external secret stores.
- Mount secrets as files at runtime.
- Use .env file (ignored in .gitignore) for local dev only.

Ways to Manage Secrets

Docker Secrets (Swarm)	Built-in secure storage, mounted as files at <code>/run/secrets/<name></code>
Environment File	Use <u>.env</u> for local, NOT in image.
External Secrets	Use tools like HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
Runtime Injection	Inject secrets during deployment (CI/CD)



TIP

Secrets should be injected at runtime, not baked at build time.



Q3: What is Docker Swarm? How is it different from Kubernetes?

A:

Docker Swarm is Docker's native orchestration tool for managing a cluster of Docker nodes. Kubernetes (K8s) is a more advanced orchestration platform with more features.

Swarm vs Kubernetes

Feature	Docker Swarm	Kubernetes
Complexity	Simple & easy	Complex
Setup	Easy (Docker CLI)	Harder
Learning Curve	Low	High
Scalability	Good	Excellent
Features	Basic (built-in)	Very Rich
Ecosystem	Smaller	Huge
Best For	Small/Medium apps	Large/Enterprise apps

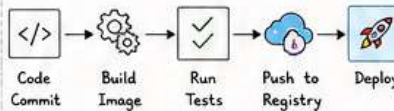
Quick Rule

Swarm = Simplicity | Kubernetes = Power

Q4: How do you use Docker in a CI/CD pipeline? Give an example.

A:

- Build image
- Run tests in container
- Push image to registry
- Deploy to staging/production



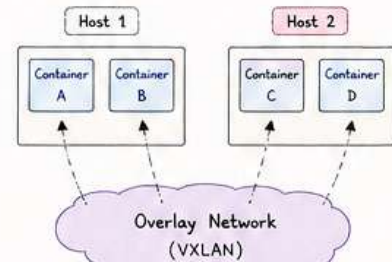
Example (GitHub Actions)

```
name: CI/CD Docker Example
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Image
        run: docker build -t myapp:${{ github.sha }}
      - name: Run Tests
        run: docker run --rm myapp npm test
      - name: Push Image
        run: |
          echo ${{ secrets.DOCKER_PASSWORD }} |
          docker login -u ${{ secrets.DOCKER_USER }} --password-stdin
          docker push myapp:${{ github.sha }}
```

Q5: What is an overlay network in Docker and when is it used?

A:

- Overlay network connects containers across multiple Docker hosts.
- It is used in Docker Swarm (multi-host communication).
- Works at Layer 3 (IP level) using VXLAN encapsulation.



Use Cases

- ✓ Multi-host communication
- ✓ Microservices across nodes
- ✓ Docker Swarm services

2/3

Q6: How do you set resource limits (CPU/Memory) on a container?

A:

- Use `--cpus`, `--memory`, `--memory-swap` flags.
- Helps in performance, stability and cost control.

Examples

```
# Limit CPU to 1.5 cores
docker run -d --name app --cpus="1.5" myapp

# Limit Memory to 512MB
docker run -d --name app --memory="512m" myapp

# Memory + Swap limit
docker run -d --name app --memory="512m" \
--memory-swap="1g" myapp
```

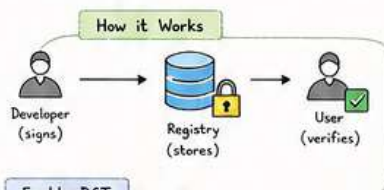
NOTE

--memory-swap = total (memory + swap)
Set it equal to memory to disable swap.

Q7: What is Docker Content Trust and how does it improve security?

A:

- Docker Content Trust (DCT) ensures the integrity and authenticity of Docker images.
- It uses digital signatures to verify that an image has not been tampered with.
- Prevents man-in-the-middle attacks and image poisoning.



Enable DCT

```
export DOCKER_CONTENT_TRUST=1
export DOCKER_CONTENT_TRUST_SERVER=https://notary.docker.io

docker push myapp:1.0
# Image is signed and verified on pull
```

Memory Trick



Secure Image + Minimal Image + Limited Resources = Strong & Fast Production System



Q8: How do you troubleshoot a container that is not working properly?

A:

Step-by-step approach:

- 1 Check Container Status
`docker ps -a`
- 2 View Logs
`docker logs <container_id>`
- 3 Inspect Container
`docker inspect <container_id>`
- 4 Execute Inside Container
`docker exec -it <container_id> sh`
- 5 Check Resource Usage
`docker stats <container_id>`
- 6 Check Events
`docker events --since 1h`
- 7 Common Issues
Port conflict, wrong config, insufficient permissions, missing dependencies.

TIP

Logs are your best friend!



KEEP PRACTICING

- Build
- Break
- Fix
- Repeat



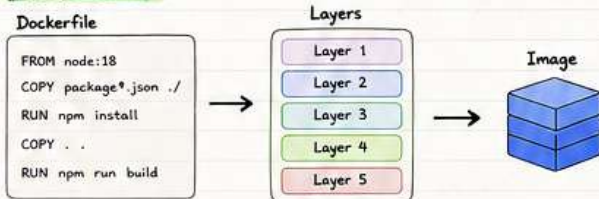
3/3

INTERVIEW QUESTIONS

1) Q: How does Layer Caching work in Docker and how can we optimize build speed?

A: Docker builds images **layer by layer**. Each instruction in the Dockerfile creates a layer. If a layer hasn't changed, Docker **reuses it from cache** instead of rebuilding.

How it works:



How to optimize build speed:

- Order Dockerfile instructions from **least** changing to **most** changing.
- Copy dependency files first (e.g., package.json) and install deps before copying the rest.
- Use `.dockerignore` to exclude unnecessary files.
- Use **multi-stage builds** to reduce final image size.
- Avoid unnecessary RUN commands.

SMART ORDER

Memory Trick
"Stable first,"
then change
the rest!"

Page 1

2) Q: What is Multi-stage Build and when do we use it?

A: Multi-stage build allows you to use multiple FROM statements in a Dockerfile. Each stage can have its own base image and you can copy **artifacts** (files) from one stage to another.

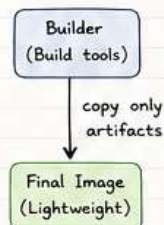
When to use it?

- To build smaller images (only keep what you need).
- To separate build environment from runtime environment.
- To keep secrets and build tools out of the final image.

Example:

```
# Stage 1: Build
FROM node:18 AS builder
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build

# Stage 2: Production
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```



Benefit:



Page 2

3) Q: What is the difference between Volumes and Bind Mounts? When to use each?

Feature	Volumes	Bind Mounts
Managed by Docker	Yes	No (managed by host)
Location	Stored in Docker's storage area	Any path on the host system
Portability	High	Lower (depends on host path)
Performance	Better on Docker Desktop (esp. Mac/Win)	Better on Linux
Use Case	Persistent data, databases, production	Dev environments, config files, source code

When to use?

- Use Volumes for data persistence (survives container removal).
- Use Bind Mounts for development (easier code changes and debugging).

Quick Note
Volumes = Docker manages
Bind Mounts = You manage



Page 3

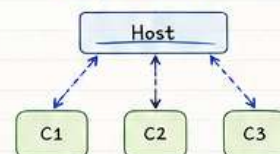
4) Q: What are the types of Docker Networks and when to use each?

Network Type	Description	When to Use
bridge	Default network for containers on the same host	Standalone containers that need to communicate
host	Container shares the host's network stack	High performance applications
none	No networking	Highly secure containers with no network access
overlay	Connects containers across multiple Docker hosts (Swarm mode)	Multi-host applications in Docker Swarm

Tip

bridge = default & simple
host = speed
none = isolation
overlay = scale across hosts

Bridge Network (example)

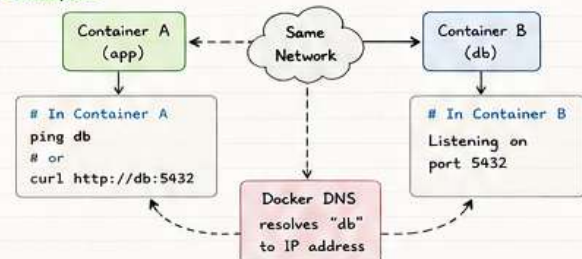


Page 4

5) Q: How does a container communicate with another container in the same network?

A: Containers in the same network can communicate using container name as a hostname. Docker provides an embedded **DNS** (Domain Name System) that resolves container names to their IP addresses.

Example:



Key Point

Use service/container name,
not IP address!

Why it's good?

- ✓ IP can change
- ✓ Names are stable
- ✓ Easier to manage

Page 5

6) Q: What is Docker Compose and when do we use it instead of docker run?

A: Docker Compose is a tool for defining and running multi-container Docker applications. You define services in a `docker-compose.yml` file and run everything with a single command.

docker run (Use when)

- Quick test
- Run a single container
- Temporary tasks

VS

Docker Compose (Use when)

- Multi-container apps
- Need networking between services
- Easier to start/stop/manage
- Environments (dev/staging/prod)



7) Q: How do you use environment variables in Docker safely?

- Use `--env` or `--env-file` instead of hardcoding in Dockerfile.
- Use `.env` file with Docker Compose (and add `.env` to `.gitignore`).
- Use **Docker Secrets** for sensitive data in production (Swarm/K8s).



8) Q: What is HEALTHCHECK in Dockerfile and how do we use it?

A: HEALTHCHECK is an instruction to check a container's health periodically. Docker uses it to determine if the container is healthy, unhealthy, or starting.

Example:

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
CMD curl -f http://localhost/health || exit 1
```

Why use it?

- ✓ Auto restart unhealthy containers
- ✓ Better monitoring
- ✓ Reliability

Page 6